

Parte 3 - Propuesta de framework

Michael Garzón · Prueba técnica Automatizador QA · Playwright + TypeScript

1. Arquitectura y carpetas

Separo el proyecto por capas. Las pruebas dicen qué se valida y las páginas y clientes dicen cómo. Si cambia un selector o un endpoint, se corrige en un solo lugar. Este repositorio ya usa esta estructura en versión reducida; en un proyecto grande agregaría components/ y utils/.

2. Patrón de diseño

Page Object Model con fixtures de Playwright para la web y Service Object (ReqresClient) para la API. Para móvil, Screen Objects con Appium. Selectores por rol, etiqueta o data-testid, nunca XPath absoluto.

```
.github/workflows/ CI
|
| api/      clientes de servicio
|
| config/   ambientes
|
| data/     datos de prueba
|
| pages/    page objects
|
| reports/  HTML, JUnit, capturas
|
| tests/
|   |
|   | api/
|   | web/
|   |   fixtures.ts
|
| azure-pipelines.yml
|
| playwright.config.ts
```

3. Datos de prueba

Datos centralizados en data/ y nombres únicos por ejecución (QA-NoExiste-<timestamp>) para que las pruebas se puedan repetir. Si se necesita un estado previo, se crea y se limpia por API.

4. Configuración por ambientes

TEST_ENV elige las URLs en config/environments.ts. Los secretos van en .env local (no se versiona) o en Variable Groups con Key Vault en el pipeline.

5. Evidencias y reportes

Reporte HTML con pasos (test.step) y JUnit publicado en Azure DevOps. Captura en cada prueba; video y trace solo cuando falla.

6. Paralelismo y pruebas inestables

Pruebas independientes con fullyParallel y --shard entre agentes. Sin pausas fijas, solo esperas por condición. Un reintento en CI: si pasa al reintentar queda marcada como flaky, se aísla con @quarantine y se corrige.

7. Git y CI/CD

Ramas feature/* con PR a main protegida por el pipeline. En Azure DevOps corre el smoke en cada PR y la regresión completa cada noche: npm ci → navegador → tsc → pruebas → publicar reportes.

Q Search tests

All 12

✓ Passed 12

Failed 0

Flaky 0

Skipped 0

🔄

⚙️

9/30/2026, 4:38:17 PM Total time: 18.9s

▼ web/login.spec.ts

✓ Escenario A - Inicio de sesion > WEB-01 Login exitoso con credenciales validas muestra el Dashboard

web-chromium web smoke

web/login.spec.ts:8

15.3s

✓ Escenario A - Casos alternos > WEB-03 Login con contrasena incorrecta muestra "Invalid credentials"

web-chromium web regression

web/login.spec.ts:38

15.6s

Reporte HTML de la última ejecución de este proyecto: 12 pruebas pasadas, con los tags que usa el pipeline.